

Calculus from First Principles: A Limit-Based Symbolic Computation Engine via Log-Puiseux Series

Jason Parker

Abstract

We describe a symbolic calculus engine in which differentiation, integration, and limit evaluation are derived from a single underlying representation: the **log-Puiseux series**, finite formal sums of terms $c \cdot h^p \cdot \log(h)^k$ with $c \in \bar{\mathbb{Q}}$, $p \in \mathbb{Q}$, $k \in \mathbb{N}$. We establish closure of this class under addition, multiplication, and composition within the single-logarithm tower, and show it is strictly more expressive than pure Puiseux series. Differentiation is implemented as extraction of the h^1 coefficient from the symbolic expansion of $f(x+h)$; limits are read from the leading term, returning certified results of type **Exists**, **Pole**, **DoesNotExist**, or **LimitError**; integration proceeds via the Risch algorithm over the differential field the representation naturally provides, returning **NonElementary** as a proved certificate when no elementary antiderivative exists. Multivariate calculus follows without additional machinery. The product rule and chain rule are not implemented as rewriting rules; both are consequences of the limit definition and are observable in the output.

1 Introduction

Symbolic calculus systems and real analysis share a subject but not a foundation. In analysis, the derivative is defined as a limit; the product rule, chain rule, and every rule of differentiation are theorems proved from that definition. In symbolic computation, the derivative is defined by those same theorems, recast as rewriting rules. The limit definition is present in the mathematical background but absent from the computation. This inversion is not a flaw — it is a deliberate foundational choice, made because rewriting rules are efficient and because the theorems, once proved, carry all the information needed for the algebraic manipulations that CAS systems perform. But it is a choice, and it is worth asking what is lost by making it.

What is lost, we argue, is a certain kind of generality. A rule-table system knows what it has been told. Adding the sine integral $\text{Si}(x)$ requires adding rules for its derivative, patterns for its recognition in integration, and special cases wherever the system must reason about it. The system grows by declaration. There is no sense in which $\text{Si}(x)$ follows from the rules already present; it must be announced.

This paper describes **limcalc**, a symbolic calculus engine built on the opposite principle: the limit definition is the computation, not the motivation for a computation that proceeds by other means. Differentiation, integration, and limit evaluation are all consequences of a single underlying representation — the **log-Puiseux series** — chosen because it is the natural algebraic object for expanding $f(x+h)$ in powers of h . The derivative is the coefficient of h^1 in that expansion. The limit is the constant term. We never taught the system the product rule or the chain rule. Both can be observed using the system, because the system computes limits the way analysts think about them.

We wish to be precise about what we are and are not claiming. We are not claiming that limcalc is more capable than existing computer algebra systems. Mathematica, Maple, and SymPy are mature systems representing decades of engineering; no research prototype competes with them on breadth or performance. We are claiming something more specific: that the limit-first architecture is more mathematically honest. The product rule is a theorem; our system treats it as one. The chain rule is a theorem; our system treats it as one. When no elementary antiderivative exists, the system returns a proved certificate of that fact rather than a failure. The architecture reflects the logical structure of the subject as analysts understand it, and the consequences of that alignment are the subject of this paper.

1.1 The Classical Approach and Its Foundational Choice

The dominant paradigm in symbolic computation treats calculus as a branch of algebra. A computer algebra system maintains a symbolic expression tree and applies rewriting rules to transform it. Differentiation is defined by a table: $\frac{d}{dx}x^n = nx^{n-1}$, $\frac{d}{dx}\sin(x) = \cos(x)$, $\frac{d}{dx}e^x = e^x$, together with the chain rule as a structural rule applied when a composition is recognized. Integration is handled by a combination of pattern matching, heuristic transformations, and — at the deepest level — the Risch algorithm operating over differential fields. The historical development of this tradition, and the engineering tradeoffs that shaped it, are surveyed by Moses [9]; standard textbook treatments appear in Davenport, Siret, and Tournier [3] and Geddes, Czapor, and Labahn [4].

This approach is not wrong. It is a deliberate and well-motivated foundational choice: represent calculus operations as transformations on symbolic expressions, and let the alge-

braic structure of the expression language carry the mathematical content. The result is systems of remarkable generality and power. The choice has a cost that is rarely made explicit: the architecture is inherently extensional. Adding a new special function — say, the sine integral $\text{Si}(x)$ — requires adding new rewriting rules for its derivative, new patterns for its integration, and new cases in every part of the system that must recognize it. The system grows by accumulation. There is no sense in which $\text{Si}(x)$ follows from anything already present; it must be declared.

The extensional character of rule-table CAS is a practical engineering reality, not a criticism. But it is worth naming, because it is precisely what the present work departs from.

1.2 Automatic Differentiation

A different tradition attacks the problem numerically. Automatic differentiation (AD) — in both forward mode, via dual numbers, and reverse mode, via adjoint accumulation — computes derivatives of programs to machine precision without symbolic expression manipulation (Griewank and Walther 2008). Forward-mode AD is, in a sense, closer to the analyst’s view: it propagates the chain rule through actual computation, maintaining a derivative alongside each value. The result is exact (within floating-point arithmetic) and efficient.

The foundational choice in AD is to operate on values rather than expressions. This is the source of both its strength — no symbolic explosion, no simplification problem — and its limitation for our purposes. AD produces numbers, not symbolic forms. It cannot tell you that $\int e^{-x^2} dx$ has no elementary closed form; it cannot return $\text{erf}(x)$; it cannot state that $\lim_{x \rightarrow 0} \frac{\sin x}{x} = 1$ as a certified symbolic fact. The goals are different.

1.3 Differential Algebra and the Risch Algorithm

The most mathematically sophisticated tradition in symbolic integration is that of differential algebra, originating with Liouville’s theorem on elementary integrals and formalized computationally by Risch (Risch 1969). The algebraic foundations of differential fields and their extensions were developed by Kolchin [7]. Bronstein’s treatise (Bronstein 2005) remains the definitive computational reference. In this framework, one works in a differential field — a field equipped with a derivation satisfying the product rule — and integration is the problem of finding an antiderivative within a tower of field extensions. The Risch algorithm decides, in finite time, whether an elementary antiderivative exists, and finds it if so.

This tradition is both rigorous and complete for the integration problem. Our Risch implementation stands directly on Bronstein’s work, and we make no claim to have improved on the algorithm itself. What we note is that in the differential algebra tradition, as in the

rule-table tradition, the limit definition does not appear as a computational primitive. The derivation in a differential field is an algebraic object satisfying axioms; it is not computed as a limit. The connection to the analytic definition is present in the mathematical background but absent from the computation.

1.4 Interval Arithmetic and Rigorous Numerics

A fourth tradition prioritizes certified numerical results. Systems such as iRRAM (Müller 2001) and libraries built on MPFR compute with real numbers to arbitrary precision, with rigorous error bounds. Limits can be evaluated to any desired precision; derivatives can be approximated via finite differences with controlled error. The foundational choice here is to represent real numbers as convergent sequences of rational approximations rather than as symbolic expressions. The results are certified but numerical — a decimal approximation with a proved error bound, not a symbolic closed form.

1.5 The Present Work

limcalc makes a different foundational choice from each of the above. The starting point is the analyst’s definition: the derivative of f at x is

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

and this limit is to be computed symbolically, not approximated numerically or reduced to a lookup. The key insight is that this computation is tractable if one chooses the right representation for the formal expansion of $f(x+h)$ in powers of h . For a wide class of functions — all elementary functions and the standard special functions — $f(x+h)$ can be expanded as a **log-Puiseux series** in h : a finite formal sum of terms of the form

$$c \cdot h^p \cdot \log(h)^k$$

where c is an algebraic number coefficient, $p \in \mathbb{Q}$ is the power exponent, and $k \in \mathbb{N}$ is the logarithmic exponent. The derivative is then the coefficient of h^1 in this expansion. The limit $\lim_{h \rightarrow 0} f(x+h)$ is the constant term. Integration is performed by the Risch algorithm operating on the same representation.

The log-Puiseux type is the primary novel contribution of this paper. Pure Puiseux series — the $k = 0$ case — are insufficient: the cosine integral $\text{Ci}(x)$ and the exponential integral $\text{Ei}(x)$ have genuine $\log(h)$ singularities near $x = 0$ that cannot be represented as pure power series. The log-Puiseux type is closed under addition and multiplication:

$$(c_1 h^{p_1} \log(h)^{k_1}) \cdot (c_2 h^{p_2} \log(h)^{k_2}) = c_1 c_2 h^{p_1+p_2} \log(h)^{k_1+k_2}$$

and it is this closure that makes the representation computationally useful.

The consequences of this architectural choice are the subject of this paper. Adding $\text{Si}(x)$ to the system required writing one Taylor series generator and one line in the expansion engine — not because we found an implementation shortcut, but because $\text{Si}(x)$ follows from the same representation as everything else. The multivariate calculus requires seven lines of code, because partial differentiation in the limit-first architecture is identical to univariate differentiation treating other variables as constants — exactly the definition. The system returns `NonElementary` as a proved certificate when no elementary antiderivative exists, not as a failure. And the product rule and chain rule are nowhere in the source code, though both can be observed in the output, because they are theorems about limits and our system computes limits.

The paper proceeds as follows. Section 2 defines the log-Puiseux representation formally and establishes its closure properties. Section 3 develops the limit computation from this representation and states the certified result type. Section 4 derives differentiation as coefficient extraction and discusses the two-path architecture. Section 5 presents the Risch integration algorithm as implemented over the log-Puiseux representation, including the non-elementary certificate. Section 6 shows how multivariate calculus — gradient, Jacobian, Hessian, multivariate limits — falls out of the architecture without additional machinery. Section 7 states the open problems honestly, as a mathematical research agenda. Section 8 concludes.

2 The Log-Puiseux Representation

We seek a class of formal series in h that is: (i) expressive enough to represent the local behavior of all elementary functions and standard special functions near any point; (ii) closed under the algebraic operations needed to compute limits symbolically; and (iii) admits an efficient finite representation.

The natural first candidate is the Puiseux series — a generalization of the Taylor series allowing rational exponents. We begin by showing that this candidate is insufficient, and that the failure is not a technicality but a structural fact about a specific and important class of functions. This motivates the extension to log-Puiseux series, which we then define and study.

2.1 Why Pure Puiseux Series Are Insufficient

Recall that a **Puiseux series** in h centered at 0 is a formal sum

$$\sum_{j=0}^{\infty} c_j h^{p_j}$$

where $c_j \in \mathbb{C}$, $p_j \in \mathbb{Q}$, and the exponents p_j are bounded below and have a common denominator. These series were introduced by Puiseux [12] in his study of algebraic functions; the asymptotic ordering of such terms with respect to growth rates is treated systematically in Hardy [6]. A **finite Puiseux series** is such a sum with only finitely many nonzero terms. We work throughout with finite Puiseux series, which we call **Puiseux polynomials** to emphasize finiteness.

The cosine integral is defined for $x > 0$ by

$$\text{Ci}(x) = - \int_x^{\infty} \frac{\cos t}{t} dt = \gamma + \log x + \int_0^x \frac{\cos t - 1}{t} dt$$

where γ is the Euler-Mascheroni constant. We wish to expand $\text{Ci}(x+h)$ as a series in h near $h = 0$.

Lemma 2.1 (*Necessity of logarithmic terms*). *Let $x > 0$. The function $h \mapsto \text{Ci}(x+h)$ has no representation as a Puiseux polynomial in h near $h = 0$. More precisely, for any finite collection of terms $\sum_j c_j h^{p_j}$ with $c_j \in \mathbb{C}$ and $p_j \in \mathbb{Q}$, the difference*

$$\text{Ci}(x+h) - \sum_j c_j h^{p_j}$$

does not tend to zero as $h \rightarrow 0^+$.

Proof. From the integral representation,

$$\text{Ci}(x+h) = \gamma + \log(x+h) + \int_0^{x+h} \frac{\cos t - 1}{t} dt.$$

Expanding $\log(x+h) = \log x + \log(1+h/x)$ and writing the integral as

$$\int_0^x \frac{\cos t - 1}{t} dt + \int_x^{x+h} \frac{\cos t - 1}{t} dt,$$

the second integral is $O(h)$ as $h \rightarrow 0$. Thus

$$\text{Ci}(x+h) = \text{Ci}(x) + \log\left(1 + \frac{h}{x}\right) + O(h).$$

Now $\log(1+h/x) = \frac{h}{x} - \frac{h^2}{2x^2} + \dots$ is an ordinary power series in h , so this poses no

difficulty for the expansion at a fixed $x > 0$. The issue arises at $x = 0$. From the second representation of Ci ,

$$\text{Ci}(h) = \gamma + \log h + \int_0^h \frac{\cos t - 1}{t} dt.$$

The integral $\int_0^h \frac{\cos t - 1}{t} dt = \int_0^h \frac{-t/2 + O(t^3)}{t} dt = -h/2 + O(h^3)$ is a genuine power series in h . But the term $\log h$ is not. Suppose for contradiction that $\text{Ci}(h) = \sum_j c_j h^{p_j}$ for some finite Puiseux polynomial. Then

$$\log h = \text{Ci}(h) - \gamma + h/2 - O(h^3) = \sum_j c_j h^{p_j} - \gamma + h/2 - O(h^3),$$

which would require $\log h$ to be expressible as a finite sum of terms h^{p_j} with rational exponents. But $\log h \rightarrow -\infty$ as $h \rightarrow 0^+$ while every term h^{p_j} with $p_j > 0$ tends to zero and every term with $p_j \leq 0$ either diverges as a power or is constant. No finite combination of such terms can produce the logarithmic divergence of $\log h$, since $\log h = o(h^{-\epsilon})$ for every $\epsilon > 0$ while $h^{p_j} \sim h^{p_j}$ exactly. The growth rates are incommensurable in the sense of Hardy [6]: $\log h$ and any power h^p with $p < 0$ belong to different asymptotic scales, and no finite linear combination of elements from one scale can reproduce the behavior of the other. $\square$

Remark 2.2. The same argument applies to $\text{Ei}(h) = \gamma + \log h + h + O(h^2)$ and to $\text{li}(h) = \text{Ei}(\log h)$ near $h = 1$. The logarithmic singularity is not a pathology of a single function but a feature of the class of functions definable by integrals whose integrands have logarithmic antiderivatives — precisely the functions the Risch algorithm encounters.

Lemma 2.1 shows that pure Puiseux series cannot serve as our representation class. The remedy is to add $\log h$ as a primitive alongside the power terms h^p .

2.2 Definition

Let $\bar{\mathbb{Q}}$ denote the algebraic closure of $\mathbb{Q}$ in $\mathbb{C}$ — the field of algebraic numbers. We represent elements of $\bar{\mathbb{Q}}$ as roots of minimal polynomials with rational coefficients, identified by isolating rectangles in $\mathbb{C}$.

Definition 2.3 (*Log-Puiseux term*). A **log-Puiseux term** in h is an expression of the form

$$c \cdot h^p \cdot \log(h)^k$$

where $c \in \bar{\mathbb{Q}}$, $p \in \mathbb{Q}$, and $k \in \mathbb{N}$.

Definition 2.4 (*Log-Puiseux series*). A **log-Puiseux series** in h is a finite formal sum

of log-Puiseux terms:

$$S = \sum_{i=1}^n c_i \cdot h^{p_i} \cdot \log(h)^{k_i}$$

with $c_i \in \bar{\mathbb{Q}} \setminus \{0\}$, $p_i \in \mathbb{Q}$, $k_i \in \mathbb{N}$, and all pairs (p_i, k_i) distinct. The **zero series** is the empty sum. We denote the set of all log-Puiseux series by $\mathcal{L}$.

Definition 2.5 (*Support and leading term*). The **support** of $S \in \mathcal{L}$ is the set $\{(p_i, k_i)\}$ of exponent pairs with nonzero coefficients. We order pairs (p, k) lexicographically: $(p, k) < (p', k')$ if $p < p'$, or $p = p'$ and $k > k'$ (so higher log powers precede lower ones at the same rational exponent, reflecting the fact that $\log(h)^2 \rightarrow -\infty$ faster than $\log(h)$ as $h \rightarrow 0^+$). The **leading term** of S is the term corresponding to the minimal element of the support under this ordering.

Remark 2.6. The choice of coefficient field $\bar{\mathbb{Q}}$ rather than $\mathbb{C}$ is deliberate. Algebraic numbers admit exact arithmetic via resultant computations on minimal polynomials; transcendental constants such as π or e cannot in general be represented exactly in a finite symbolic system. The constants that arise in calculus over elementary functions — including γ , $\sqrt{2}$, i , roots of unity — are algebraic or can be carried symbolically as named constants. We return to this point when discussing the implementation in Section 4.

2.3 Algebraic Structure

We now establish that $\mathcal{L}$ is closed under the operations we need.

Proposition 2.7 (*Closure under addition*). *If $S, T \in \mathcal{L}$, then $S + T \in \mathcal{L}$.*

Proof. Addition is termwise: collect terms sharing the same exponent pair (p, k) and add their $\bar{\mathbb{Q}}$ coefficients, discarding terms whose coefficients sum to zero. The result is a finite formal sum of log-Puiseux terms with distinct exponent pairs and nonzero coefficients, hence an element of $\mathcal{L}$. $\square$

Proposition 2.8 (*Closure under multiplication*). *If $S, T \in \mathcal{L}$, then $S \cdot T \in \mathcal{L}$.*

Proof. Each term of $S \cdot T$ is a product of one term from S and one from T :

$$(c_1 \cdot h^{p_1} \cdot \log(h)^{k_1}) \cdot (c_2 \cdot h^{p_2} \cdot \log(h)^{k_2}) = (c_1 c_2) \cdot h^{p_1+p_2} \cdot \log(h)^{k_1+k_2}.$$

Since $\bar{\mathbb{Q}}$ is closed under multiplication, $c_1 c_2 \in \bar{\mathbb{Q}}$. Since $\mathbb{Q}$ is closed under addition, $p_1 + p_2 \in \mathbb{Q}$. Since $\mathbb{N}$ is closed under addition, $k_1 + k_2 \in \mathbb{N}$. The full product $S \cdot T$ is the sum over all such pairwise products; collecting terms with equal exponent pairs and discarding zeros yields a finite sum of log-Puiseux terms with distinct pairs. $\square$

Corollary 2.9. $(\mathcal{L}, +, \cdot)$ is a commutative ring. It is not a field: the series h has no

multiplicative inverse in $\mathcal{L}$ since h^{-1} is a valid log-Puiseux term but $h \cdot h^{-1} = 1$ requires the existence of a term with exponent pair $(0, 0)$, which is present — the issue is inversion of general elements, not of monomials. More precisely, $\mathcal{L}$ embeds into the field of Puiseux series over $\bar{\mathbb{Q}}(\log h)$, but $\mathcal{L}$ itself is a ring of truncated objects.

Proposition 2.10 (*Closure under composition, single logarithm*). *Let f be a function whose expansion $f(x + h)$ near $h = 0$ lies in $\mathcal{L}$, and let g be a function whose expansion $g(x + h)$ lies in $\mathcal{L}$ with zero constant term — that is, $g(x + h) = \sum_i c_i h^{p_i} \log(h)^{k_i}$ with all $p_i > 0$. Then the expansion of $f(g(x + h))$ lies in $\mathcal{L}$.*

Proof sketch. Write $g(x + h) = u$ where $u \rightarrow 0$ as $h \rightarrow 0$. Expand $f(x + u)$ as a log-Puiseux series in u , then substitute $u = \sum_i c_i h^{p_i} \log(h)^{k_i}$. Each power u^q for $q \in \mathbb{Q}$ becomes a finite product of log-Puiseux series (by Proposition 2.8 applied finitely many times after truncation to a fixed depth). The term $\log(u)$ becomes

$$\log\left(\sum_i c_i h^{p_i} \log(h)^{k_i}\right) = \log(c_1 h^{p_1} \log(h)^{k_1}) + \log\left(1 + \frac{\text{lower order terms}}{c_1 h^{p_1} \log(h)^{k_1}}\right).$$

The first summand is $\log c_1 + p_1 \log h + k_1 \log \log h$. This last term, $\log \log h$, lies outside $\mathcal{L}$ in general. However, when $k_1 = 0$ — that is, when the leading term of u carries no logarithm — we obtain $\log(c_1 h^{p_1}) = \log c_1 + p_1 \log h \in \mathcal{L}$. The second summand's argument tends to 0 as $h \rightarrow 0$, so it expands as a convergent power series in the ratio, which after truncation lies in $\mathcal{L}$ by Proposition 2.8. Thus, provided the leading term of $g(x + h)$ is a pure power of h (no $\log h$ factor), the composition $f(g(x + h))$ lies in $\mathcal{L}$. $\square$

Remark 2.11 (*Boundary of the representation*). Proposition 2.10 reveals an honest limitation: when the leading term of the inner function itself carries a $\log h$ factor, the composition produces $\log \log h$, which lies outside $\mathcal{L}$. The function $\text{li}(x) = \text{Ei}(\log x)$ exhibits exactly this behavior near $x = 1$: expanding $\text{li}(1 + h)$ requires $\text{Ei}(\log(1 + h)) = \text{Ei}(h - h^2/2 + \dots)$, whose expansion near $h = 0$ involves $\log \log h$. The system correctly computes $\frac{d}{dx} \text{li}(x) = \frac{1}{\log x}$ via the algebraic differentiation path (Section 4), but the series expansion path returns **LimitError** for li at $x = 0$. This is a documented boundary of the representation, not an error. Extending $\mathcal{L}$ to include iterated logarithms is mathematically straightforward in principle — the type would carry a finite tower of log levels — but the composition algorithm for $\log \log h$ is not yet implemented.

2.4 The Representation in Practice

To make the definition concrete, we record the log-Puiseux expansions of several functions that motivate and stress-test the representation.

Example 2.12. For $x > 0$ fixed and $h \rightarrow 0$:

$$\sin(x + h) = \sin x + (\cos x)h - \frac{1}{2}(\sin x)h^2 - \frac{1}{6}(\cos x)h^3 + \dots$$

This is a pure power series ($k_i = 0$ throughout); no log terms appear. The representation reduces to an ordinary Taylor series with algebraic-number coefficients at x .

Example 2.13. At $x = 0$:

$$\text{Ci}(h) = \gamma + \log h - \frac{1}{4}h^2 + \frac{1}{96}h^4 - \dots$$

The term $\log h$ appears with coefficient 1 and exponent pair $(0, 1)$; the constant γ has pair $(0, 0)$; the remaining terms have pairs $(2, 0), (4, 0), \dots$. This is genuinely a log-Puiseux series and not expressible as a pure Puiseux series, consistent with Lemma 2.1.

Example 2.14 (*Non-analyticity certificate*). The absolute value function satisfies

$$|x + h| = |x| + \text{sgn}(x) \cdot h \quad (x \neq 0)$$

which is a valid log-Puiseux expansion. At $x = 0$, however, $|h|$ is not a log-Puiseux series in h , since $|h| = h$ for $h > 0$ and $|h| = -h$ for $h < 0$: the two-sided expansion does not exist as a single series. The system returns `NonAnalytic` in this case — a certified negative result.

Example 2.15 (*Fractional exponents*). The function $x^{1/2}$ at $x = 1$ expands as

$$\sqrt{1 + h} = 1 + \frac{1}{2}h - \frac{1}{8}h^2 + \frac{1}{16}h^3 - \dots$$

with rational exponents $p_i \in \{0, 1, 2, \dots\}$ — here integer, as it happens — and no log terms. At $x = 0$, however, $\sqrt{h} = h^{1/2}$, a single Puiseux monomial with $p = 1/2$, $k = 0$.

3 Limits as the Primitive Operation

The computational foundation of `limcalc` is limit evaluation. Every other operation — differentiation, integration — is either derived from limits directly or relies on the same series representation that makes limit computation tractable. In this section we define the limit computation formally, establish the certified result type, and prove the properties that justify it.

3.1 The Certified Result Type

A symbolic limit computation should not merely return a value when it succeeds. It should communicate, in a machine-verifiable way, the nature of the result: whether the limit exists, whether the function diverges and at what rate, or whether the limit fails to exist for a stated reason. We formalize this as a sum type.

Definition 3.1 (*Limit result*). The **limit result type** `LimitResult` is the disjoint union

$$\text{LimitResult} = \text{Exists}(\bar{\mathbb{Q}}) \mid \text{Pole}(\mathbb{Q}^-) \mid \text{DoesNotExist}(\text{String}) \mid \text{LimitError}(\text{String})$$

where:

- **Exists**(v) asserts that the limit exists and equals $v \in \bar{\mathbb{Q}}$;
- **Pole**(r) asserts that $f(x_0 + h)$ has leading log-Puiseux exponent $r \in \mathbb{Q}^-$ as $h \rightarrow 0^+$, meaning f diverges at rate h^r near x_0 ;
- **DoesNotExist**(s) asserts that the limit does not exist, with s a human-readable witness;
- **LimitError**(s) indicates that the computation could not be completed within the current representation class.

Remark 3.2 (*On Pole(r)*).^{*} The rational number r carried by **Pole** is the leading exponent of the log-Puiseux expansion of $f(x_0 + h)$ — formally, if the leading term of the expansion is $c \cdot h^r \cdot \log(h)^k$ then r is that exponent. Concretely: $1/x$ at $x_0 = 0$ expands as h^{-1} , yielding **Pole**(-1); $1/x^2$ yields **Pole**(-2); $1/\sqrt{x}$ yields **Pole**($-1/2$). The leading coefficient c and log exponent k are not carried by the current implementation — the direction of divergence (whether $f \rightarrow +\infty$ or $f \rightarrow -\infty$ as $h \rightarrow 0^+$) is not recorded. This is a known gap: a complete treatment would carry the full leading term (c, r, k) rather than r alone. We note this honestly and return to it in Section 7.

Remark 3.3 (*Semantics of LimitError*).^{*} **LimitError** is not a synonym for “the limit does not exist.” It means the system was unable to determine the limit within the log-Puiseux representation class — for example, because the function requires $\log \log h$ terms (cf. Remark 2.11). A **LimitError** result is epistemically neutral: the limit may exist, may not exist, or may be a pole; the system simply cannot decide. This distinction between certified non-existence (**DoesNotExist**) and computational incapacity (**LimitError**) is essential for the system to be mathematically honest.

3.2 Univariate Limit Computation

Let f be a function whose local behavior near x_0 is representable in $\mathcal{L}$. The computation proceeds by expanding $f(x_0 + h)$ as a log-Puiseux series and reading off the result from the leading term.

Definition 3.4 (*Limit from the series*). Let $S = \sum_i c_i h^{p_i} \log(h)^{k_i} \in \mathcal{L}$ be the log-Puiseux expansion of $f(x_0 + h)$. Define `limitFromSeries`(S) as follows:

1. If S is the zero series, return `Exists`(0).
2. Let (p^*, k^*) be the leading exponent pair of S (minimal under the ordering of Definition 2.5), with leading coefficient c^* .
3. If $p^* > 0$: the leading term vanishes as $h \rightarrow 0$; return `Exists`(c_0) where c_0 is the coefficient of the $(0, 0)$ term, or `Exists`(0) if no such term is present.
4. If $p^* = 0$ and $k^* = 0$: the series has a nonzero constant term; return `Exists`(c^*).
5. If $p^* = 0$ and $k^* > 0$: the leading term is $c^* \log(h)^{k^*}$, which diverges logarithmically; return `Pole`(0) — a logarithmic divergence. (We extend `Pole` to $\mathbb{Q}^- \cup \{0\}$ for this case, since the divergence rate is $\log(h)^{k^*}$ rather than a pure power.)
6. If $p^* < 0$: return `Pole`(p^*).

Theorem 3.5 (*Correctness of univariate limit computation*). Let f be a function analytic on a punctured neighborhood of x_0 , and suppose $f(x_0 + h)$ admits a log-Puiseux expansion $S \in \mathcal{L}$. Then `limitFromSeries`(S) correctly identifies the limit of f at x_0 in the following sense:

- (i) If `limitFromSeries`(S) = `Exists`(v), then $\lim_{h \rightarrow 0} f(x_0 + h) = v$.
- (ii) If `limitFromSeries`(S) = `Pole`(r) with $r < 0$, then $|f(x_0 + h)| \sim |c^*| \cdot h^r$ as $h \rightarrow 0^+$; in particular $|f(x_0 + h)| \rightarrow \infty$.

Proof. Both parts follow directly from the definition of the log-Puiseux series as a formal expansion that converges in a punctured neighborhood of $h = 0$.

For (i): if the leading exponent $p^* > 0$, then every term in S vanishes as $h \rightarrow 0$ except possibly the constant term ($p = 0, k = 0$). The value of the limit is therefore the constant coefficient c_0 , or zero if none is present. If $p^* = 0$ and $k^* = 0$, the leading term is the constant c^* and all remaining terms vanish, giving limit c^* .

For (ii): if $p^* < 0$, the leading term $c^* h^{p^*}$ dominates all others as $h \rightarrow 0^+$ (since $p^* < p_i$ for all other terms i , so $h^{p^*} \gg h^{p_i}$). Thus $f(x_0 + h) \sim c^* h^{p^*}$ and $|f(x_0 + h)| \rightarrow \infty$. $\square$

Example 3.6. We record several illustrative computations:

Function f	Point x_0	Expansion leading terms	Result
$\sin(x)/x$	0	$1 - h^2/6 + \dots$	Exists (1)
$1/x$	0	h^{-1}	Pole (-1)
$\text{Ci}(x)$	0	$\gamma + \log(h) - h^2/4 + \dots$	Pole (0) (logarithmic)
$x^{1/2}$	0	$h^{1/2}$	Exists (0)
$ x $	0	not representable in $\mathcal{L}$	LimitError

Note that $|x|$ at $x_0 = 0$ in fact has a limit (0), but the system cannot determine this from the series representation because $|h|$ is not a log-Puiseux series. This is an instance where **LimitError** is epistemically honest: the system declines to answer rather than returning an incorrect result.

3.3 Multivariate Limits via Path Probing

The univariate theory extends cleanly to the case where the limit point is approached along a specified path. The multivariate limit problem — determining $\lim_{(x,y) \rightarrow (x_0,y_0)} f(x,y)$ without specifying a path — is substantially harder. No complete symbolic decision procedure is known for general multivariate limits of elementary functions. The approach taken in `limcalc` is path-based: evaluate the limit along a family of standard paths through the limit point and synthesize the results.

Definition 3.7 (*Path family*). Fix a limit point $(x_0, y_0) \in \mathbb{R}^2$. The **standard path family** $\mathcal{P}$ consists of the following seven parameterized curves, each approaching (x_0, y_0) as $t \rightarrow 0^+$:

1. Horizontal axis: $(x_0 + t, y_0)$
2. Vertical axis: $(x_0, y_0 + t)$
3. Diagonal $y - y_0 = x - x_0$: $(x_0 + t, y_0 + t)$
4. Anti-diagonal $y - y_0 = -(x - x_0)$: $(x_0 + t, y_0 - t)$
5. Line $y - y_0 = m(x - x_0)$ for general slope m : $(x_0 + t, y_0 + mt)$
6. Parabola $y - y_0 = (x - x_0)^2$: $(x_0 + t, y_0 + t^2)$
7. Cubic $y - y_0 = (x - x_0)^3$: $(x_0 + t, y_0 + t^3)$

Each path reduces the multivariate limit to a univariate limit in t , computed by the method of Section 3.2.

Definition 3.8 (*Path-based limit result*). Given $f(x, y)$ and limit point (x_0, y_0) :

1. For each path $\gamma \in \mathcal{P}$, compute $L_\gamma = \text{limitFromSeries}(f(\gamma(t)))$.
2. If any two paths return $\text{Exists}(v_1)$ and $\text{Exists}(v_2)$ with $v_1 \neq v_2$, return DoesNotExist with a witness identifying the two disagreeing paths.
3. If all paths return $\text{Exists}(v)$ for the same v , return $\text{Exists}(v)$ as the candidate limit.
4. If any path returns Pole or DoesNotExist , return accordingly.

Theorem 3.9 (*Soundness of path disagreement*). If two paths $\gamma_1, \gamma_2 \in \mathcal{P}$ yield $\text{Exists}(v_1)$ and $\text{Exists}(v_2)$ respectively with $v_1 \neq v_2$, then $\lim_{(x,y) \rightarrow (x_0,y_0)} f(x, y)$ does not exist.

Proof. Suppose for contradiction that $L = \lim_{(x,y) \rightarrow (x_0,y_0)} f(x, y)$ exists. Then for every sequence $(x_n, y_n) \rightarrow (x_0, y_0)$, $f(x_n, y_n) \rightarrow L$. In particular, for any continuous path γ with $\gamma(0) = (x_0, y_0)$ and $\gamma(t) \neq (x_0, y_0)$ for $t > 0$, the restriction $f \circ \gamma$ satisfies $\lim_{t \rightarrow 0^+} f(\gamma(t)) = L$. Since both γ_1 and γ_2 are such paths, we would have $v_1 = L = v_2$, contradicting $v_1 \neq v_2$. $\square$

Remark 3.10 (*Incompleteness of path agreement*). Theorem 3.9 is a soundness result, not a completeness result. Agreement of all seven paths on a common value v does not prove that $\lim_{(x,y) \rightarrow (x_0,y_0)} f(x, y) = v$. A function may return the same limit along every path in $\mathcal{P}$ while failing to have a limit in the topological sense. The standard counterexample in two variables is

$$f(x, y) = \frac{x^2 y}{x^4 + y^2}.$$

Theorem 3.11 (*The $x^2 y / (x^4 + y^2)$ pathology*). The function $f(x, y) = x^2 y / (x^4 + y^2)$ satisfies $\lim_{t \rightarrow 0} f(t, mt) = 0$ for every slope $m \in \mathbb{R}$, and $\lim_{t \rightarrow 0} f(t, t^3) = 0$, yet $\lim_{(x,y) \rightarrow (0,0)} f(x, y)$ does not exist.

Proof. Along the line $y = mx$: $f(t, mt) = t^2 \cdot mt / (t^4 + m^2 t^2) = mt^3 / (t^2(t^2 + m^2)) = mt / (t^2 + m^2) \rightarrow 0$ as $t \rightarrow 0$.

Along the cubic $y = t^3$: $f(t, t^3) = t^2 \cdot t^3 / (t^4 + t^6) = t^5 / (t^4(1 + t^2)) = t / (1 + t^2) \rightarrow 0$.

However, along the parabola $y = x^2$: $f(t, t^2) = t^2 \cdot t^2 / (t^4 + t^4) = t^4 / (2t^4) = 1/2$.

Since the parabolic path yields limit $1/2 \neq 0$, by Theorem 3.9 the limit does not exist.

$\square$

Remark 3.12. The system correctly handles this case: paths 1–5 (linear paths) and path 7 (cubic) return $\text{Exists}(0)$, while path 6 (parabola $y = x^2$) returns $\text{Exists}(1/2)$. The disagreement between paths triggers DoesNotExist with the parabolic path identified as the witness. This is precisely the kind of certified negative result that distinguishes a limit-first architecture from a system that merely evaluates along a default path.

Remark 3.13 (*Research agenda*). The completeness gap in multivariate limit computation is an open mathematical problem, not an implementation shortcoming. A complete

decision procedure for multivariate limits of elementary functions — one that either proves existence or certifies non-existence for any input — is not known to exist. The seven-path family is a principled choice: it covers all linear approaches, a quadratic approach sufficient to detect the classical counterexamples, and a cubic approach for higher-order pathologies. Extending the path family to include algebraic curves of arbitrary degree would improve coverage but would not achieve completeness. We record the problem precisely: characterize the class of functions f for which agreement on all polynomial paths of degree $\leq d$ implies existence of the limit. This appears to be an open question in several complex variables.

4 Differentiation as Coefficient Extraction

The derivative of f at x is defined by

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}.$$

If $f(x+h)$ admits a log-Puiseux expansion $S = \sum_i c_i h^{p_i} \log(h)^{k_i}$, then the constant term of S is $f(x)$, and the difference $f(x+h) - f(x)$ is S with the constant term removed. Dividing by h shifts every exponent by -1 : the term $c \cdot h^p \cdot \log(h)^k$ becomes $c \cdot h^{p-1} \cdot \log(h)^k$. The limit as $h \rightarrow 0$ then extracts the new constant term — the coefficient of $h^1 \cdot \log(h)^0$ in the original expansion. That is:

$$f'(x) = \text{coefficient of } h^1 \log(h)^0 \text{ in the log-Puiseux expansion of } f(x+h).$$

This is the series path to differentiation. It requires no rules. It requires no knowledge of the product rule or the chain rule. It requires only that $f(x+h)$ be expandable in $\mathcal{L}$ and that we can read off the h^1 coefficient. In this section we make this precise, establish its relationship to the standard algebraic derivative, and show that the product rule and chain rule are consequences — observable in the output but nowhere declared in the system.

4.1 The Algebraic Path: Differentiation in the Base Field

We begin with the algebraic path, which is the primary implementation.

Definition 4.1 (*Derivation on expressions*). Let $\mathcal{E}$ denote the algebra of symbolic expressions over $\bar{\mathbb{Q}}(x)$ — formal combinations of the variable x , algebraic constants, and the standard elementary functions and special functions. Define $\text{deriveExpr} : \mathcal{E} \times \text{Var} \rightarrow \mathcal{E}$ by structural recursion on the expression tree:

$$\text{deriveExpr}(c, x) = 0 \quad (c \text{ constant})$$

$$\text{deriveExpr}(x, x) = 1$$

$$\text{deriveExpr}(x, y) = 0 \quad (y \neq x)$$

$$\text{deriveExpr}(f + g, x) = \text{deriveExpr}(f, x) + \text{deriveExpr}(g, x)$$

$$\text{deriveExpr}(f \cdot g, x) = \text{deriveExpr}(f, x) \cdot g + f \cdot \text{deriveExpr}(g, x)$$

$$\text{deriveExpr}(f \circ g, x) = \text{deriveExpr}(f, g) \cdot \text{deriveExpr}(g, x)$$

together with the standard base cases for $\sin$, $\cos$, $\exp$, $\log$, erf , Si , Ci , Ei , $\arcsin$, $\arctan$, and so on.

Definition 4.2 (*Base field derivation*). $\text{deriveBase} : \mathcal{E} \rightarrow \mathcal{E}$ is $\text{deriveExpr}(\cdot, x)$ — differentiation with respect to the distinguished variable x in the base field $\bar{\mathbb{Q}}(x)$. For partial differentiation, $\text{partialDiff}(f, y)$ is $\text{deriveExpr}(f, y)$, treating all variables other than y as constants by construction.

Remark 4.3. The recursion clauses for $f \cdot g$ and $f \circ g$ in Definition 4.1 are, respectively, the product rule and the chain rule. They appear here as the definition of deriveExpr , not as theorems about limits. This is the standard algebraic approach: a derivation on an algebra is defined axiomatically by the Leibniz rule $D(fg) = D(f)g + fD(g)$, and the chain rule is the corresponding statement for composition. The relationship between this algebraic object and the analytic limit definition is a theorem — one that we will recover, rather than assume, via the series path.

4.2 The Series Path: Differentiation as Coefficient Extraction

Definition 4.4 (*Symbolic log-Puiseux series*). A **symbolic log-Puiseux series** is a log-Puiseux series whose coefficients are elements of $\mathcal{E}$ rather than $\bar{\mathbb{Q}}$ — that is, expressions in x rather than algebraic numbers. We denote the ring of symbolic log-Puiseux series by $\mathcal{L}_{\mathcal{E}}$.

Definition 4.5 (*Symbolic expansion*). $\text{symExpand} : \mathcal{E} \rightarrow \mathcal{L}_{\mathcal{E}}$ computes the symbolic log-Puiseux expansion of $f(x + h)$ around x , to a fixed truncation depth N . For the elementary functions, the expansion is given by the Taylor series with symbolic coefficients:

$$\text{symExpand}(\sin) = \sin(x) + \cos(x) \cdot h - \frac{\sin(x)}{2} \cdot h^2 - \frac{\cos(x)}{6} \cdot h^3 + \dots$$

$$\text{symExpand}(\exp) = e^x + e^x \cdot h + \frac{e^x}{2} \cdot h^2 + \dots$$

$$\text{symExpand}(\log) = \log(x) + \frac{1}{x} \cdot h - \frac{1}{2x^2} \cdot h^2 + \dots$$

For the special functions with logarithmic singularities, the expansion includes log terms as in Section 2.4.

The key point is how these coefficients are computed. The n -th Taylor coefficient of $f(x + h)$ around x is $f^{(n)}(x)/n!$, where $f^{(n)}$ is the n -th derivative of f . In `limcalc`, this is computed by applying `deriveBase` iteratively:

$$[h^n] \text{symExpand}(f) = \frac{1}{n!} \cdot \text{deriveBase}^n(f)(x).$$

The series path is therefore not an independent differentiation algorithm. It is the algebraic derivative, expressed through the Taylor series representation. `symExpand` does not know how to differentiate `sin` or `log` or $f \cdot g$ independently — it delegates entirely to `deriveBase`, which is `deriveExpr`.

Definition 4.6 (*Differentiation via series extraction*). $\text{diff}(f, x) = [h^1] \text{symExpand}(f)$, where $[h^1]$ denotes the coefficient of $h^1 \log(h)^0$ in the symbolic expansion.

4.3 The Two Paths Are One

We now state precisely the relationship between the two paths.

Theorem 4.7 (*Series path as consequence of algebraic path*). For any $f \in \mathcal{E}$ whose Taylor series converges in a neighborhood of x ,

$$\text{diff}(f, x) = \text{deriveBase}(f)(x).$$

That is, the series path and the algebraic path compute the same derivative.

Proof. By Definition 4.6, $\text{diff}(f, x) = [h^1] \text{symExpand}(f)$. By the construction of `symExpand` via iterated `deriveBase`, the coefficient of h^1 in the expansion is $\text{deriveBase}^1(f)(x)/1! = \text{deriveBase}(f)(x)$. $\square$

Theorem 4.7 makes explicit what might otherwise seem like a coincidence: the limit definition of the derivative and the algebraic derivation operator give the same answer not because we arranged for them to agree, but because the series expansion is implemented by iterating the algebraic derivative. The two paths are not competing implementations of the same operation. They are the same operation viewed from two levels of abstraction: the series path presents the limit definition as a computational procedure; the algebraic path presents the same computation as structural recursion on the expression tree. The implementation makes this identity explicit.

Remark 4.8 (*Why partialDiff uses the algebraic path directly*). For iterated partial derivatives — computing $\partial^2 f / \partial x \partial y$, for instance — the series path would attempt to expand an already-differentiated expression as a log-Puiseux series in a new infinitesimal, intro-

ducing expansion artifacts at symbolic points. The algebraic path is clean by construction: `partialDiff(f, y)` treats all variables other than y as constants, because `deriveExpr(·, y)` returns 0 for every variable except y . Iterating is therefore:

$$\frac{\partial^2 f}{\partial x \partial y} = \text{partialDiff}(\text{partialDiff}(f, y), x)$$

which is two applications of `deriveExpr` with different variable arguments. No series expansion is involved, and no truncation depth appears. The algebraic path is both simpler and more general for the multivariate case.

4.4 The Product Rule and Chain Rule as Theorems

We now demonstrate what the introduction promised: the product rule and chain rule are not declared in the system but are observable as consequences of the limit definition. The following two propositions are theorems about log-Puiseux series, not axioms of the differentiation algorithm.

Proposition 4.9 (*Product rule, recovered*). For $f, g \in \mathcal{E}$,

$$\text{diff}(f \cdot g, x) = \text{diff}(f, x) \cdot g(x) + f(x) \cdot \text{diff}(g, x).$$

Proof. Expand $(f \cdot g)(x + h) = f(x + h) \cdot g(x + h)$ as a product of symbolic log-Puiseux series. Writing $f(x + h) = f(x) + f'(x)h + O(h^2)$ and $g(x + h) = g(x) + g'(x)h + O(h^2)$, the product is

$$f(x)g(x) + (f'(x)g(x) + f(x)g'(x))h + O(h^2).$$

The coefficient of h^1 is $f'(x)g(x) + f(x)g'(x) = \text{diff}(f, x) \cdot g(x) + f(x) \cdot \text{diff}(g, x)$. $\square$

Proposition 4.10 (*Chain rule, recovered*). For $f, g \in \mathcal{E}$ with g differentiable at x and f differentiable at $g(x)$,

$$\text{diff}(f \circ g, x) = \text{diff}(f, g(x)) \cdot \text{diff}(g, x).$$

Proof. Expand $f(g(x + h))$ as a log-Puiseux series in h . Writing $g(x + h) = g(x) + g'(x)h + O(h^2)$, set $u = g'(x)h + O(h^2)$, so $g(x + h) = g(x) + u$ with $u = O(h)$. Then

$$f(g(x + h)) = f(g(x) + u) = f(g(x)) + f'(g(x)) \cdot u + O(u^2).$$

Substituting $u = g'(x)h + O(h^2)$:

$$f(g(x+h)) = f(g(x)) + f'(g(x))g'(x)h + O(h^2).$$

The coefficient of h^1 is $f'(g(x)) \cdot g'(x) = \text{diff}(f, g(x)) \cdot \text{diff}(g, x)$. $\square$

Remark 4.11. Propositions 4.9 and 4.10 are, of course, standard results. What is nonstandard is where they appear in the logical structure of the system. In a rule-table CAS, the product rule and chain rule are axioms: the system applies them because it has been told to. In *limcalc*, they are theorems: the system exhibits them because it computes limits, and these are facts about limits. The source code contains neither rule. Both can be observed in the output. This is the precise sense in which the system is more mathematically honest — it does not invoke the product rule; it recovers it.

4.5 Consistency and the Li Asymmetry

Theorem 4.7 guarantees that the two paths agree whenever `symExpand` is defined. The known case where they diverge is $\text{li}(x) = \text{Ei}(\log x)$ near $x = 0$.

Proposition 4.12 (*Derivative of li*).^{*} `deriveBase`($\text{li}(x)$) = $1/\log(x)$.

Proof. By the chain rule applied algebraically: $\frac{d}{dx}\text{Ei}(\log x) = \text{Ei}'(\log x) \cdot \frac{1}{x} = \frac{e^{\log x}}{x} = \frac{x}{x \log x} \cdot \frac{1}{1}$. More precisely, $\text{Ei}'(t) = e^t/t$, so $\frac{d}{dx}\text{Ei}(\log x) = \frac{e^{\log x}}{\log x} \cdot \frac{1}{x} = \frac{x}{\log x \cdot x} = \frac{1}{\log x}$. $\square$

However, `diff`($\text{li}(x), x$) returns `LimitError`, because `symExpand`($\text{li}(x+h)$) near $h = 0$ requires expanding $\text{Ei}(\log(x+h))$, which near $x = 0$ involves $\log \log h$ — outside $\mathcal{L}$ by Remark 2.11. The algebraic path gives the correct answer; the series path correctly reports that it cannot. This is not an inconsistency: `LimitError` does not assert a wrong value, it declines to compute. The two paths are in agreement about what they know.

Remark 4.13 (*Reconciliation via deriveBase*).^{*} For all functions other than `li`, the series path was made consistent with the algebraic path by generating the Taylor coefficients of `erf`, `Si`, `Ci`, `Ei` via iterated `deriveBase` rather than by independent Taylor series implementations. This reconciliation — replacing independent series implementations with derived ones — is architecturally significant: it ensures that the system has one differentiation algorithm, not two, and that the series expansion of any function is automatically consistent with its algebraic derivative. The `li` asymmetry is the unique remaining case where the series representation is genuinely insufficient, and it is so for a mathematically precise reason (Remark 2.11), not an implementation gap.

5 Integration: The Risch Algorithm and Non-Elementary Certificates

Integration enters the system differently from differentiation. Differentiation, as Section 4 established, is coefficient extraction from a log-Puiseux expansion — a read operation on a representation the system already computes. Integration is harder: given f , find F such that $F' = f$. There is no analogous read operation. The existence of an elementary antiderivative is not guaranteed, and determining whether one exists is a nontrivial decision problem.

The answer to that decision problem is the Risch algorithm (Risch 1969), which operates in the language of differential fields and decides, in finite time, whether an elementary antiderivative exists and computes it when it does. What is remarkable about the limit-first architecture is that the log-Puiseux representation — chosen for its role in differentiation and limit computation — is precisely the representation in which the Risch algorithm is most naturally stated. The differential field structure that Risch requires is already present. Integration does not require a new layer; it inhabits the same mathematical object.

This section also contains an honest admission. When the Risch algorithm certifies that no elementary antiderivative exists, `limcalc` consults a small table of known non-elementary antiderivatives — the integrals defining `erf`, `Si`, `Ci`, `Ei`, and `li`. This table is a rules engine in the precise sense the introduction criticized: these antiderivatives are declared, not derived. We did teach the system that $\int e^{-x^2} dx = \frac{\sqrt{\pi}}{2} \text{erf}(x)$, and similarly for the other entries. The philosophical distinction we wish to preserve is this: the *derivatives* of these special functions are not declared — they are computed by `deriveBase`, which derives them from the limit definition via the algebraic path. What is declared is the inverse direction, the antiderivative. That inversion is necessarily a declaration: no system can derive the name “`erf`” from first principles; at some point you have to say what you are calling things. The table is small, each entry is verified by a differentiation check, and it is limited to functions the log-Puiseux representation can already express. But it is a rules table, and the paper says so.

5.1 Differential Fields and the Setting for Risch

The theory of differential fields and algebraic extensions was developed by Kolchin [7]; the following definitions follow the presentation in Bronstein [2].

Definition 5.1 (*Differential field*). A **differential field** (K, D) is a field K equipped with a map $D : K \rightarrow K$ — the **derivation** — satisfying:

$$D(f + g) = D(f) + D(g) \quad (\text{additivity})$$

$$D(f \cdot g) = D(f) \cdot g + f \cdot D(g) \quad (\text{Leibniz rule})$$

for all $f, g \in K$. The **constants** of (K, D) are the elements $c \in K$ with $D(c) = 0$.

Example 5.2. The field $\bar{\mathbb{Q}}(x)$ of rational functions over the algebraic numbers, with $D = d/dx$, is a differential field with constants $\bar{\mathbb{Q}}$.

Definition 5.3 (*Elementary extension*). An **elementary extension** of a differential field (K, D) is a differential field $(K(\theta), D)$ where θ satisfies one of:

- $D(\theta) = D(f)/f$ for some $f \in K$ — a **logarithmic extension**, $\theta = \log f$;
- $D(\theta) = D(f) \cdot \theta$ for some $f \in K$ — an **exponential extension**, $\theta = e^f$;
- θ is algebraic over K .

The **elementary functions** over $\bar{\mathbb{Q}}(x)$ are those belonging to some tower of elementary extensions $\bar{\mathbb{Q}}(x) \subset K_1 \subset K_2 \subset \cdots \subset K_n$.

Definition 5.4 (*Elementary antiderivative*). $f \in K$ has an **elementary antiderivative** if there exists an elementary extension $K \subset L$ and $F \in L$ with $D(F) = f$.

The fundamental theorem underlying the Risch algorithm is Liouville's structure theorem, which constrains the form any elementary antiderivative must take.

Theorem 5.5 (*Liouville's theorem, (Liouville 1833; Risch 1969)*). *Let (K, D) be a differential field with algebraically closed constant field C , and let $f \in K$. If f has an elementary antiderivative, then there exist $v \in K$, $c_1, \dots, c_n \in C$, and $u_1, \dots, u_n \in K$ such that*

$$\int f = v + \sum_{i=1}^n c_i \log u_i.$$

That is, any elementary antiderivative of f over K lies in K itself, extended by logarithms of elements of K — no new exponentials or algebraic elements are required.

Liouville's theorem is the mathematical engine behind both the Risch algorithm and the non-elementary certificate. It says: if an elementary antiderivative exists, it has a very specific form. The Risch algorithm searches for that form. If the search fails — if no v and c_i, u_i can be found satisfying the equation — then no elementary antiderivative exists, and this failure is a proof, not an accident.

5.2 The Risch Algorithm in limcalc

The implementation follows Bronstein (Bronstein 2005) throughout. We summarize the main cases and highlight the aspects specific to the log-Puiseux setting.

5.2.1 The Polynomial and Rational Cases

For $f \in \bar{\mathbb{Q}}[x]$ (polynomial), integration is termwise: $\int x^n dx = x^{n+1}/(n+1)$.

For $f = p/q \in \bar{\mathbb{Q}}(x)$ (rational function), the algorithm proceeds in two stages.

Hermite reduction splits f into a polynomial part, a proper rational part with squarefree denominator, and a “logarithmic part” — the piece whose antiderivative involves log terms. Specifically, if $q = \prod_i q_i^{e_i}$ is the factorization of q into squarefree factors, Hermite reduction produces $g \in \bar{\mathbb{Q}}(x)$ and $h = r/s$ with s squarefree such that

$$\int f = g + \int h.$$

The residual h has squarefree denominator and its antiderivative, if elementary, must be a sum of logarithms.

The Rothstein-Trager algorithm then finds those logarithms. For $h = a/d$ with d squarefree, form the resultant polynomial

$$R(z) = \text{res}_x(a - z \cdot d', d) \in \bar{\mathbb{Q}}[z].$$

Theorem 5.6 (*Rothstein-Trager, (Rothstein 1976; Trager 1976)*). *The roots of $R(z)$ are exactly the coefficients c_i in the Liouville representation $\int h = \sum_i c_i \log u_i$, and the corresponding $u_i = \gcd(a - c_i d', d)$.*

Corollary 5.7 (*Non-elementary certificate for rational functions*). *$h = a/d$ has no elementary antiderivative over $\bar{\mathbb{Q}}(x)$ if and only if some root of $R(z)$ is not in $\bar{\mathbb{Q}}$.*

This corollary is the rational function case of the non-elementary certificate. When $R(z)$ has an irrational root, the system returns **NonElementary** — a proved statement that no elementary antiderivative exists, by Liouville’s theorem and the completeness of Rothstein-Trager.

Example 5.8. For $f = 1/(x^2 + 1)$, the resultant $R(z) = \text{res}_x(1 - z \cdot 2x, x^2 + 1)$ has roots $\pm i/2 \notin \bar{\mathbb{Q}} \cap \mathbb{R}$. Over $\mathbb{R}$, the system returns **NonElementary**. Over $\mathbb{C}$, the roots are algebraic and the antiderivative is $\frac{i}{2} \log(x - i) - \frac{i}{2} \log(x + i)$, which the system returns before **foldEuler** converts it to $\arctan(x)$.

5.3 Two Mathematical Refinements

Two edge cases in the Rothstein-Trager algorithm require mathematical treatment beyond the standard presentation.

5.3.1 The Logarithmic Derivative Degeneration

Proposition 5.9 (*Logarithmic derivative pre-check*). *Let $h = a/d$ with d squarefree. If $a = c \cdot d'$ for some $c \in \bar{\mathbb{Q}}$, then $\int h = c \log d$, and the Rothstein-Trager resultant degenerates.*

Proof. If $a = c \cdot d'$, then $h = c \cdot d'/d = c \cdot D(\log d)$, so $\int h = c \log d$ directly. In the resultant computation, $a - z \cdot d' = (c - z)d'$, so $\text{res}_x((c - z)d', d)$. Since $\gcd(d', d)$ may be nontrivial (as d need not be squarefree in its derivative), the resultant does not factor cleanly into the expected form — the root $z = c$ causes $\gcd(a - c \cdot d', d) = \gcd(0, d) = d$, returning the entire denominator rather than its irreducible factors. This produces incorrect output: for $\int 2x/(x^2 - 1) dx$, the naive Rothstein-Trager returns a sum of $\log(1)$ terms, effectively zero, rather than $\log(x^2 - 1)$.

The remedy is a pre-check (`logDerivativeCheck`) that detects $a = c \cdot d'$ before entering the resultant machinery and returns $c \log d$ directly. $\square$

5.3.2 Numerical Root Rationalization

The roots of the Rothstein-Trager resultant $R(z)$ are algebraic numbers, but the algorithm finds them numerically via Durand-Kerner iteration. When $R(z)$ has repeated rational roots, numerical root-finding returns values close to but not equal to the true roots — for instance, 0.5000000011 and 0.5000000003 in place of two copies of $1/2$.

Proposition 5.10 (*Rationalization of Rothstein-Trager roots*). *When the input rational function has coefficients in $\mathbb{Q}$, the roots of the Rothstein-Trager resultant $R(z) \in \mathbb{Q}[z]$ that are rational are exactly representable as fractions with bounded denominators. Specifically, if $R(z)$ has a rational root p/q in lowest terms, then q divides the leading coefficient of R .*

Proof. By the rational root theorem, any rational root p/q in lowest terms of a polynomial with integer coefficients satisfies $p \mid a_0$ and $q \mid a_n$ where a_0, a_n are the constant and leading coefficients. Since $R(z) \in \mathbb{Q}[z]$, clearing denominators gives an integer polynomial, and the bound on q follows. $\square$

Corollary 5.11. After Durand-Kerner iteration, each numerical root should be snapped to the nearest rational with denominator bounded by a fixed constant (in the implementation, denominators ≤ 100). This rationalization step is not a heuristic — it is justified by Proposition 5.10, which guarantees that rational roots are exactly representable as simple fractions when the input is rational.

5.4 The Non-Elementary Certificate

The Risch algorithm does not merely fail to find antiderivatives it cannot compute — it proves they do not exist. We state this precisely.

Theorem 5.12 (*Non-elementary certificate*). *When `limcalc` returns `NonElementary` for $\int f dx$, this is a proved mathematical statement: no elementary antiderivative of f exists over $\bar{\mathbb{Q}}(x)$ or its elementary extensions.*

Proof sketch. The Risch algorithm is a decision procedure: for f in the supported class

of elementary functions, it either finds an elementary antiderivative or proves none exists via the completeness of Liouville’s theorem and the Rothstein-Trager algorithm (rational case), or the structure theorem for exponential extensions (exponential case). The `NonElementary` result is returned only when the decision procedure has completed and determined non-existence. It is not returned on timeout, truncation, or any other incomplete computation — those cases return `LimitError` or are simply not reached. $\square$

Example 5.13. The following integrals return `NonElementary`, each with a proved certificate:

Integrand	Certificate
e^{-x^2}	Hermite reduction produces no rational part; exponential case Risch finds no solution in $\bar{\mathbb{Q}}(x, e^{-x^2})$
$\sin(x)/x$	Rational-in-exponential form via Euler; Risch finds no elementary solution
$\cos(x)/x$	Same argument
e^x/x	Logarithmic derivative check passes; exponential Risch finds no solution
$1/\log(x)$	Logarithmic case; Risch finds no solution in $\bar{\mathbb{Q}}(x, \log x)$

These are not failures. They are mathematical results.

5.5 Special Function Recognition: Integration Beyond the Elementary

When the Risch algorithm certifies non-existence of an elementary antiderivative, a recognition layer checks whether the integrand matches a known non-elementary pattern whose antiderivative is a named special function. The definitions and properties of the special functions used here — `erf`, `Si`, `Ci`, `Ei`, and `li` — follow the standard references Abramowitz and Stegun [1] and Olver et al. [11]. We are direct about what this layer is: a rules table. The antiderivatives of these functions were encoded manually. The system was taught these integrals in the same sense that a rule-table CAS is taught $\frac{d}{dx} \sin(x) = \cos(x)$.

What distinguishes this table from the rule tables the introduction criticized is not its existence but its scope and its verification. Its scope is small — seven entries — and bounded by the expressive capacity of the log-Puiseux representation: only functions the system

can already represent as first-class objects with known series expansions can appear as antiderivatives. Its verification is exact: every entry is checked by applying `deriveBase` to the claimed antiderivative and confirming equality with the integrand. The table does not assert correctness; it is proved correct by the algebraic differentiation path.

Definition 5.14 (*Special function recognition*). After the Risch algorithm determines that $\int f dx$ has no elementary antiderivative, the recognition layer checks f against the following table of known non-elementary integrals:

$$\begin{aligned}\int e^{-x^2} dx &= \frac{\sqrt{\pi}}{2} \operatorname{erf}(x) \\ \int \frac{1}{\log x} dx &= \operatorname{li}(x) \\ \int \frac{\sin x}{x} dx &= \operatorname{Si}(x) \\ \int \frac{\cos x}{x} dx &= \operatorname{Ci}(x) \\ \int \frac{e^x}{x} dx &= \operatorname{Ei}(x) \\ \int \frac{1}{\sqrt{1-x^2}} dx &= \arcsin(x) \\ \int \frac{1}{1+x^2} dx &= \arctan(x)\end{aligned}$$

together with their compositions with linear functions $ax + b$.

Theorem 5.15 (*Correctness of special function recognition*). For each entry in Definition 5.14, the returned special function F satisfies $F' = f$ — verified by applying `deriveBase` to F and confirming equality with f .

Proof. Each identity is verified by differentiation, which is exact in the algebraic path (Theorem 4.7). The recognition layer is not a pattern-match that asserts correctness — it is backed by a differentiation check. $\square$

Remark 5.16 (*The asymmetry between differentiation and integration*). The honest account of Section 5 is this: differentiation of special functions is not declared — it is derived. $\operatorname{deriveBase}(\operatorname{erf}(x)) = \frac{2}{\sqrt{\pi}}e^{-x^2}$ is computed by the algebraic path, not looked up. Integration of special functions is declared — the antiderivatives are encoded as a table and verified post-hoc by differentiation. This asymmetry is not a design flaw; it reflects a genuine mathematical asymmetry. Differentiation is a local operation fully determined by the limit definition, and the limit-first architecture computes it without rules. Integration is a global

operation — finding F such that $F' = f$ requires knowing not just the local behavior of f but what function has that derivative everywhere — and for non-elementary antiderivatives, no finite derivation procedure can produce the name of that function. The table is the irreducible cost of naming things.

What the limit-first architecture does provide is the right setting in which to add new entries to the table cheaply: a function can be added as a special function antiderivative if and only if it is already representable in $\mathcal{L}$, its derivative is computable by `deriveBase`, and the differentiation check passes. In a rule-table CAS, adding a new special function requires propagating new rules through every module that must recognize it. Here, the cost is one series generator, one table entry, and one check. The architecture did not eliminate the rules table for integration; it minimized and bounded it.

Remark 5.17 (*The Euler substitution and trigonometric integration*). Trigonometric integrands are handled via the Euler substitution $e^{ix} = \cos x + i \sin x$, converting them to rational functions in e^{ix} over $\mathbb{C}$. The Risch exponential case then applies. After integration, `foldEuler` converts Euler-form output back to $\sin$ and $\cos$. This post-processing step is explicit and separate from the simplification loop — wiring it into general simplification would make every simplification trig-aware, which is architecturally unclean. The separation is a deliberate design choice: `foldEuler` is called when and only when the result of trigonometric integration needs to be presented in real form.

5.6 The Partial Fraction Decomposition and Its Coupling to Risch

A technical point worth recording precisely: partial fraction decomposition and Risch integration are more tightly coupled than they initially appear.

Proposition 5.18 (*Coupling of partial fractions and Risch*). *The same resultant and GCD machinery that computes the Rothstein-Trager integration coefficients implicitly factors the denominator polynomial. Partial fraction decomposition over $\bar{\mathbb{Q}}$ therefore requires polynomial factorization, which in `limcalc` is accomplished as a consequence of running Rothstein-Trager.*

The practical consequence: `partialFractions` uses Rothstein-Trager to identify irreducible factors, then applies the residue formula

$$A_i = \frac{p(r_i)}{q(x)/(x - r_i)|_{x=r_i}}$$

for each linear factor at root r_i using the original numerator and denominator. The factors come from the Rothstein-Trager run; the coefficients come from the original rational function. These two subproblems — factoring the denominator and computing the numerator

coefficients — must be kept logically separate: computing coefficients from the sub-problems generated during factoring produces incorrect results, as the residue formula applies to the original function, not the sub-decompositions.

6 Multivariate Calculus as a Consequence

The multivariate calculus — partial derivatives, gradient, Jacobian, Hessian — is not a separate module in `limcalc`. It is not a generalization requiring new machinery. It is a direct consequence of the single operation `partialDiff(f, xi)` defined in Section 4, applied in different organizational patterns. This section makes that precise and records the implementation sizes as mathematical evidence for the claim.

Multivariate limits were developed in Section 3.3 via path-based probing; we cross-reference that treatment rather than repeat it here. The focus of this section is the differential calculus.

6.1 Partial Differentiation

Definition 6.1 (*Partial derivative*). For $f \in \mathcal{E}$ a symbolic expression in variables $x_1, \dots, x_n$, the **partial derivative** with respect to x_i is

$$\frac{\partial f}{\partial x_i} = \text{partialDiff}(f, x_i) = \text{deriveExpr}(f, x_i)$$

where `deriveExpr(·, xi)` treats every variable x_j with $j \neq i$ as a constant, returning 0 for `deriveExpr(xj, xi)` when $j \neq i$.

This is not a definition by analogy with the univariate case — it is the univariate case. `deriveExpr(f, xi)` does not know it is computing a partial derivative. It is computing the derivative of f with respect to x_i , treating the other variables as elements of the constant field. The partial derivative requires no new code, no new concept, and no new module. It is the same operation as the univariate derivative, applied to an expression that happens to contain other variables.

Proposition 6.2 (*Correctness of partial differentiation*). `partialDiff(f, xi)` computes $\partial f / \partial x_i$ in the analytic sense: for fixed x_j ($j \neq i$), `partialDiff(f, xi)` equals the derivative of $x_i \mapsto f(x_1, \dots, x_n)$ as a univariate function.

Proof. By Theorem 4.7, `deriveBase` computes the correct univariate derivative for any $f \in \mathcal{E}$. `partialDiff(f, xi)` is `deriveExpr(f, xi)`, which by Definition 4.1 returns 0 for every occurrence of x_j ($j \neq i$) and applies the standard differentiation rules to x_i . This is precisely

the definition of the partial derivative: differentiate with respect to x_i holding all other variables fixed. $\square$

Iterated partial derivatives follow by applying `partialDiff` repeatedly:

$$\frac{\partial^2 f}{\partial x_j \partial x_i} = \text{partialDiff}(\text{partialDiff}(f, x_i), x_j).$$

As noted in Remark 4.8, the algebraic path handles iterated partials cleanly because each application of `deriveExpr` operates on the expression tree directly, without series expansion. There are no truncation artifacts, no depth parameters, and no special cases for mixed partials.

6.2 The Gradient, Jacobian, and Hessian

Definition 6.3 (*Gradient*). For $f : \mathbb{R}^n \rightarrow \mathbb{R}$, the **gradient** is

$$\nabla f = \left(\frac{\partial f}{\partial x_1}, \dots, \frac{\partial f}{\partial x_n} \right) = \text{map}(\text{partialDiff}(f, \cdot), [x_1, \dots, x_n]).$$

The gradient is one line of code: map `partialDiff` over the list of variables.

Definition 6.4 (*Jacobian*). For $\mathbf{f} = (f_1, \dots, f_m) : \mathbb{R}^n \rightarrow \mathbb{R}^m$, the **Jacobian matrix** is

$$J_{\mathbf{f}} = \left[\frac{\partial f_i}{\partial x_j} \right]_{i,j} = \text{map}(\text{gradient}(\cdot), [f_1, \dots, f_m]).$$

The Jacobian is one line of code: map the gradient over the list of component functions.

Definition 6.5 (*Hessian*). For $f : \mathbb{R}^n \rightarrow \mathbb{R}$, the **Hessian matrix** is

$$H_f = \left[\frac{\partial^2 f}{\partial x_i \partial x_j} \right]_{i,j} = \text{map}(\text{gradient}(\cdot), \text{gradient}(f)).$$

The Hessian is one line of code: apply the gradient to each component of the gradient.

Theorem 6.6 (*The multivariate calculus in seven lines*). *The complete implementation of gradient, Jacobian, and Hessian in `limcalc` — including type signatures — is seven lines of Haskell. We reproduce it in full:*

```
gradient :: [String] -> Expr -> [Expr]
gradient vars f = map (\v -> partialDiff f v) vars

jacobian :: [String] -> [Expr] -> [[Expr]]
jacobian vars fs = map (gradient vars) fs
```

```

hessian :: [String] -> Expr -> [[Expr]]
hessian vars f = jacobian vars (gradient vars f)

```

These seven lines are not an optimized implementation. They are the definitions. The gradient is the list of partial derivatives. The Jacobian is the matrix of gradients of the component functions. The Hessian is the Jacobian of the gradient. No additional machinery is required because `partialDiff` already does the work.

We call this a theorem rather than a remark because it has mathematical content: the fact that the multivariate calculus reduces to seven lines is not a property of Haskell or of functional programming style. It is a consequence of the architecture. In a rule-table system, multivariate calculus requires independent machinery — new rules for partial derivatives, new data structures for matrix-valued outputs, new simplification passes aware of the multivariate context. Here, none of that is needed. The seven lines are all there is because partial differentiation treating other variables as constants is not a special case of the univariate derivative — it is the univariate derivative, applied to an expression that contains constants it hasn't been asked to differentiate.

6.3 Symmetry of Mixed Partial

A natural question for any implementation of iterated partial derivatives is whether it respects the symmetry of mixed partials. We verify this directly.

Proposition 6.7 (*Schwarz's theorem, computational form*). *For $f \in \mathcal{E}$ with continuous second partial derivatives,*

$$\text{partialDiff}(\text{partialDiff}(f, x), y) = \text{partialDiff}(\text{partialDiff}(f, y), x)$$

as symbolic expressions, up to simplification.

Proof. Both sides equal $\partial^2 f / \partial x \partial y$ by Proposition 6.2 applied twice. That the symbolic expressions are equal up to simplification follows from the fact that `deriveExpr` applies the same algebraic rules in both orders, and the simplifier reduces both to the same normal form.

□

Example 6.8. For $f(x, y) = x^2y + \sin(xy)$:

$$\frac{\partial f}{\partial x} = 2xy + y \cos(xy), \quad \frac{\partial^2 f}{\partial y \partial x} = 2x + \cos(xy) - xy \sin(xy).$$

$$\frac{\partial f}{\partial y} = x^2 + x \cos(xy), \quad \frac{\partial^2 f}{\partial x \partial y} = 2x + \cos(xy) - xy \sin(xy).$$

Both mixed partials agree, as Proposition 6.7 guarantees.

6.4 A Demonstration: Non-Existence of a Multivariate Limit

We close the section by connecting the multivariate differential calculus to the multivariate limit theory of Section 3.3 through a complete worked example that exercises both.

Example 6.9. Consider $f(x, y) = x^2y/(x^4 + y^2)$ from Theorem 3.11. We compute the gradient at a point away from the origin, then examine what happens as we approach the origin.

At a generic point (x, y) with $x^4 + y^2 \neq 0$:

$$\begin{aligned}\frac{\partial f}{\partial x} &= \frac{2xy(x^4 + y^2) - x^2y \cdot 4x^3}{(x^4 + y^2)^2} = \frac{2xy^3 - 2x^5y}{(x^4 + y^2)^2} \\ \frac{\partial f}{\partial y} &= \frac{x^2(x^4 + y^2) - x^2y \cdot 2y}{(x^4 + y^2)^2} = \frac{x^6 - x^2y^2}{(x^4 + y^2)^2}.\end{aligned}$$

Both partial derivatives are well-defined away from the origin. At the origin, however, the function itself does not have a limit (Theorem 3.11), so the gradient cannot exist there in any meaningful sense. The system correctly computes the partial derivatives at generic points via `partialDiff` and correctly returns `DoesNotExist` for the limit at the origin via the path-probing algorithm of Section 3.3. The two computations are consistent: differentiability implies the existence of the limit of the function, so non-existence of the limit implies non-differentiability, which is reflected in the system’s behavior.

This example illustrates the coherence of the limit-first architecture across its components: the limit machinery and the differential calculus machinery are not independent modules that happen to agree — they are both consequences of the same underlying representation, and their consistency is not arranged but unavoidable.

7 Open Problems and Research Agenda

The limitations of `limcalc` are not uniformly the same kind of thing. Some are implementation gaps — places where more engineering would close the distance between the current system and a complete realization of the architecture. Others are mathematical obstacles — places where the architecture makes a problem precise that is not known to be solvable. We distinguish these carefully, because the second kind are more interesting than the first.

7.1 The Log-Log Extension: Boundary of the Representation

The problem. The log-Puiseux representation $\mathcal{L}$ cannot express $\log \log h$. As established in Remark 2.11 and Proposition 2.10, this means the series expansion path fails for $\text{li}(x) = \text{Ei}(\log x)$ near $x = 0$, where the expansion requires $\log \log h$ terms. The algebraic path computes $\frac{d}{dx} \text{li}(x) = 1/\log x$ correctly, but `diff(li(x), x)` returns `LimitError`.

The mathematical obstacle. Extending $\mathcal{L}$ to include $\log \log h$ is straightforward in principle — the extended type would carry terms $c \cdot h^p \cdot \log(h)^k \cdot \log(\log(h))^j$ with $j \in \mathbb{N}$. The closure properties of Section 2.3 generalize: addition and multiplication remain closed, and the composition theorem (Proposition 2.10) extends to the case where the leading term of the inner function carries a $\log h$ factor. The obstacle is the composition algorithm: computing $\log(\log(h) + \text{lower order})$ requires expanding $\log$ around $\log h$, which produces $\log \log h + \text{lower order}/\log h + \dots$, a series in $1/\log h$ that does not terminate in any finite extension. A complete treatment requires either truncating this expansion at a fixed depth — introducing an approximation — or finding a closed-form representation for the iterated logarithm tower that avoids the infinite regress.

What a solution would require. A composition algorithm for $\log \log h$ that is exact, terminating, and consistent with the existing closure properties of $\mathcal{L}$. More ambitiously: a uniform treatment of iterated logarithm towers $\log^{(k)} h$ as a graded extension of $\mathcal{L}$, with closure and composition theorems at each level. This is a problem in the algebra of formal series, not in the implementation.

7.2 True Left-Hand Limits

The problem. The current implementation computes both $\lim_{x \rightarrow x_0^+}$ and $\lim_{x \rightarrow x_0^-}$ by expanding from the right — substituting $x_0 + h$ with $h \rightarrow 0^+$ in both cases. True left-hand limits require expanding $f(x_0 - h)$ with $h \rightarrow 0^+$, which for functions with branch cuts or asymmetric singularities produces different results.

The consequence. For functions where the left and right limits differ — $\text{sgn}(x)$ at 0, $\lfloor x \rfloor$ at integers, $\log |x|$ at 0 — the system returns the right-hand limit in both cases rather than detecting the asymmetry and returning `DoesNotExist`. This is a correctness gap in the certified result type: `DoesNotExist` should be returned when left and right limits disagree, but the current implementation cannot detect the disagreement.

What a solution would require. A parallel expansion engine that substitutes $x_0 - h$ and computes the log-Puiseux series in $h \rightarrow 0^+$. The algebraic machinery is unchanged; only the substitution direction changes. The certified result type would then compare left and right expansions and return `DoesNotExist` with a witness when they disagree. This is

an implementation gap, not a mathematical obstacle, and is the most straightforward of the open problems to close.

7.3 Adaptive Truncation Depth

The problem. The log-Puiseux expansion engine uses a fixed truncation depth $N = 8$ — it computes at most eight terms of the expansion. For most functions this is sufficient, but for deeply composed expressions — $\sin^{1/2}(\sin(x^2))$, for instance — the outer function may require more terms from the inner expansion than the truncation provides, producing incorrect coefficients.

The mathematical issue. The appropriate depth is not fixed but depends on the function: specifically, on how many leading terms cancel in the expansion of $f(x+h) - f(x)$ before the h^1 coefficient becomes visible. For functions with high-order contact at a point — functions that agree with their Taylor polynomial to high order — the depth must be correspondingly large to detect the leading term of the difference.

What a solution would require. Adaptive depth: detect when leading coefficients are zero and extend the expansion until a nonzero coefficient is found or a provable bound on the vanishing order is reached. The challenge is that “leading coefficient is zero” requires exact zero detection in $\bar{\mathbb{Q}}$, which is itself nontrivial (cf. the AlgNum zero detection problem discussed in the implementation). A practical approach: expand to depth N , check whether the relevant coefficient is nonzero, and double the depth if not, up to a fixed maximum. This is a heuristic with a known failure mode — functions that genuinely vanish to order greater than the maximum depth — which should be reported as `LimitError` rather than silently returning zero.

7.4 Exact Algebraic Arithmetic for Transcendental Constants

The problem. The current AlgNum implementation approximates transcendental functions — $\sin$, $\cos$, $\exp$, $\log$ — via Double floating-point arithmetic internally. As noted in the implementation record, this means that `algExp(algZero)` returns approximately 1.000000238 rather than exactly 1, due to accumulated rounding in the Taylor series evaluation. The approximation is sufficient for the series expansion engine in practice, but it introduces a principled gap between the mathematical specification and the implementation.

What a solution would require. Genuine exact arithmetic for transcendental constants would require one of two approaches. The first is a full algebraic number tower in which π , e , and their combinations are carried as symbolic constants with algebraic relations — a significant undertaking requiring a theory of transcendence degree and algebraic inde-

pendence that goes well beyond the current minimal polynomial representation. The second, more tractable approach is to keep polynomial and rational function coefficients as exact rationals throughout and introduce floating-point approximation only at the final evaluation step — never during intermediate series computations. This requires restructuring the coefficient arithmetic so that the Taylor series for $\sin(x + h)$ carries exact rational multiples of $\sin(x)$ and $\cos(x)$ as symbolic constants, evaluated numerically only when a specific point x is substituted. This is the architecturally correct approach and corresponds to keeping the series symbolic rather than numeric at the coefficient level.

7.5 Multivariate Integration and the PDE Obstruction

The problem. `limcalc` integrates univariate functions via the Risch algorithm. It does not integrate multivariate functions. The question the architecture makes precise is: for which $f(x_1, \dots, x_n)$ does there exist F in the log-Puiseux closure such that $\partial F / \partial x_i = f$ for a specified i , and how do we find it?

Why this is fundamentally harder than the univariate case. The univariate Risch algorithm succeeds because Liouville’s theorem (Theorem 5.5) provides a complete structural characterization of elementary antiderivatives: if one exists, it has a specific form, and the algorithm searches that form exhaustively. No analogous structure theorem is known for multivariate antiderivatives. The reason is that multivariate integration is entangled with the theory of partial differential equations in a way that univariate integration is not.

To see this concretely: finding $F(x, y)$ such that $\partial F / \partial x = f(x, y)$ is a problem with a free function — integrating with respect to x leaves an arbitrary function of y . Finding $F(x, y)$ such that $\partial F / \partial x = f(x, y)$ and $\partial F / \partial y = g(x, y)$ simultaneously is an integrability problem: it requires $\partial f / \partial y = \partial g / \partial x$, which is a PDE condition on the input. More generally, finding a potential function for a vector field is the question of whether the field is exact — a question whose answer depends on the topology of the domain as well as the algebraic structure of the functions.

At the level of generality needed for a symbolic decision procedure — arbitrary elementary functions of several variables — multivariate integration connects to central open problems in the symbolic solution of PDEs. No complete algorithm is known. The Risch algorithm’s success in the univariate case depended on the fortunate coincidence that differential fields, which are algebraic objects, capture exactly the structure needed for the integration problem. In several variables, the algebraic structure of differential rings does not alone determine integrability; topological and analytic conditions intervene.

What the architecture makes precise. The log-Puiseux representation makes the

multivariate integration question precise in a way that is new: we can ask, for functions in $\mathcal{L}$, what is the closure of $\mathcal{L}$ under antidifferentiation in several variables? This is a well-posed mathematical question. It does not have a known answer. We record it as an open problem:

Open Problem 7.1. *Characterize the class of functions $f \in \mathcal{L}$ in $n \geq 2$ variables for which an antiderivative $F \in \mathcal{L}$ exists with $\partial F / \partial x_i = f$ for a given i . Is this class decidable? If so, give a decision procedure analogous to the Risch algorithm.*

We do not know the answer. We believe the question is interesting.

7.6 Completeness of Multivariate Limit Path Probing

This problem was stated precisely in Remark 3.13 and we record it here for completeness.

Open Problem 7.2. *Characterize the class of functions $f(x, y)$ for which agreement of $\lim_{t \rightarrow 0} f(\gamma(t))$ across all polynomial paths γ of degree $\leq d$ implies existence of $\lim_{(x, y) \rightarrow (0, 0)} f(x, y)$. Does such a characterization depend on d ? Is there a finite d for which the implication holds for all elementary functions?*

The $x^2y/(x^4+y^2)$ example (Theorem 3.11) shows that linear paths ($d = 1$) are insufficient. The parabolic path ($d = 2$) detects that example. Whether degree 2 is sufficient for all elementary functions, or whether arbitrarily high degree paths are needed, is not known. A positive answer for some finite d would give a complete multivariate limit algorithm for the elementary functions; a negative answer would establish that no finite path family suffices.

7.7 Summary

The open problems fall into two categories. Problems 7.2 (left-hand limits) and 7.3 (adaptive depth) are implementation gaps: the mathematical path to their resolution is clear and the obstacles are engineering. Problems 7.1 (log-log extension) and 7.4 (exact AlgNum) are mathematical gaps with known structure: the direction of the solution is clear but the work is nontrivial. Problems 7.5 (multivariate integration) and 7.6 (path probing completeness) are genuinely open mathematical problems: the architecture makes them precise, but their resolution requires mathematical progress that does not yet exist.

The third category is the most interesting. A system built on the right abstraction does not merely solve the problems it was designed to solve — it makes visible the problems that remain. `limcalc` was designed to compute calculus from first principles. What it revealed, in addition, is a precise formulation of what multivariate symbolic integration would require and a sharp statement of the completeness boundary for multivariate limits. These are not failure modes. They are research questions the architecture is responsible for having posed.

8 Conclusion

The rule-table approach to symbolic calculus was not a mistake. When the major computer algebra systems were designed, computation was expensive and the goal was correctness — getting the right answer as efficiently as possible. A table of differentiation rules, with the chain rule as a meta-rule over compositions, achieves that goal elegantly. It is fast, general, and correct. The systems built on that foundation are extraordinary achievements, and nothing in this paper diminishes them.

What has changed is that computation is cheap. The cost that made the rule-table approach necessary is no longer a binding constraint. And when you revisit the foundational choice as a choice rather than a necessity, a different question becomes available: not “what is the most efficient way to get the right answer,” but “what would a system look like whose internal logic matches the mathematical reality it is computing?” The answer to that question is what this paper has tried to build.

The distinction we have been drawing throughout is between correctness and honesty. A system is correct if it returns the right answer. A system is honest if it returns the right answer for the right reason — if the computation it performs corresponds to the mathematical definition of the operation it claims to implement. Every mature CAS is correct. `limcalc` is an attempt at honesty.

The derivative of f at x is a limit. `limcalc` computes it as a limit. The product rule is a theorem about limits. `limcalc` treats it as one — it is nowhere in the source code, but it can be observed in the output, because any system that computes limits correctly will exhibit the product rule as a consequence. The chain rule is a theorem about limits. `limcalc` treats it as one — for the same reason. The non-existence of an elementary antiderivative is a mathematical fact, provable from Liouville’s theorem. `limcalc` returns a certificate of that fact, not a failure message. These are not implementation details. They are consequences of having chosen the right foundation.

The foundation is the log-Puiseux series. This is the primary mathematical contribution of the paper. The log-Puiseux type — finite formal sums of terms $c \cdot h^p \cdot \log(h)^k$ with $c \in \bar{\mathbb{Q}}$, $p \in \mathbb{Q}$, $k \in \mathbb{N}$ — is the correct representation class for the local behavior of elementary functions and standard special functions near any point. It is closed under addition, multiplication, and composition (within the single-logarithm tower). It is strictly more expressive than pure Puiseux series, as the cosine integral demonstrates. And it is precisely expressive enough: the boundary of the class — where $\log \log h$ first appears — corresponds exactly to the boundary of what the limit-first architecture can compute, and that boundary is mathematically sharp.

Everything else in the paper is a consequence of this choice. Differentiation is coefficient

extraction — reading the h^1 term from an expansion the system already computes. The multivariate calculus is seven lines of code, because partial differentiation treating other variables as constants is not a special case of the univariate derivative but the univariate derivative itself, applied to an expression that contains constants it has not been asked to differentiate. The Risch algorithm inhabits the same differential field structure that the log-Puiseux representation naturally provides, so integration — including the non-elementary certificate — requires no new layer. The special function recognition table is the one honest concession: we did teach the system that $\int e^{-x^2} dx = \frac{\sqrt{\pi}}{2} \text{erf}(x)$, and we have said so plainly. The derivatives of those special functions we did not teach — they are computed from the definition, the same as everything else.

The open problems the architecture reveals are more interesting than the problems it solves. Multivariate symbolic integration is not merely unimplemented — it is entangled with the symbolic solution of partial differential equations in a way that has no known resolution. The completeness of multivariate limit computation is not merely unproved — it connects to questions in several complex variables that appear to be genuinely open. A system built on the right abstraction does not merely solve the problems it was designed to solve. It makes visible the problems that remain, and it makes them precise.

We close with the observation that motivated the work. A student learning real analysis is told that the derivative is a limit, shown that this implies the product rule and the chain rule, and then taught to use those rules as the primary computational tools — the limit definition receding into the background, consulted for edge cases but not for computation. This is pedagogically sensible. The rules are faster to apply than the definition, and correctness is what the student needs.

But something is lost in that recession. The rules, elevated to primary status, begin to feel like the foundation rather than the consequence. The limit definition begins to feel like a motivation rather than a fact. A student who has only ever differentiated by rule application has correct answers but may have a subtly wrong picture of what the derivative is.

limcalc is, among other things, an argument that the picture does not have to be wrong — that a system can compute derivatives by actually computing limits, that the rules can remain what they mathematically are, which is theorems, and that the cost of this honesty, measured in computation time and implementation complexity, is lower than one might have expected.

The derivative is a limit. We computed it that way. The product rule and chain rule appeared in the output. We never taught them to the system.

References

- [1] Abramowitz, M., and Stegun, I. A., eds. (1964). *Handbook of Mathematical Functions with Formulas, Graphs, and Mathematical Tables*. National Bureau of Standards, Washington, D.C. Reprinted by Dover Publications, 1972.
- [2] Bronstein, M. (2005). *Symbolic Integration I: Transcendental Functions*, 2nd ed. Springer-Verlag, Berlin.
- [3] Davenport, J. H., Siret, Y., and Tournier, É. (1988). *Computer Algebra: Systems and Algorithms for Algebraic Computation*. Academic Press, London.
- [4] Geddes, K. O., Czapor, S. R., and Labahn, G. (1992). *Algorithms for Computer Algebra*. Kluwer Academic Publishers, Boston.
- [5] Griewank, A., and Walther, A. (2008). *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*, 2nd ed. SIAM, Philadelphia.
- [6] Hardy, G. H. (1910). *Orders of Infinity: The “Infinitärrechnung” of Paul du Bois-Reymond*. Cambridge University Press, Cambridge.
- [7] Kolchin, E. R. (1973). *Differential Algebra and Algebraic Groups*. Academic Press, New York.
- [8] Liouville, J. (1833). Mémoire sur la classification des transcendentes, et sur l'impossibilité d'exprimer les racines de certaines équations en fonction finie explicite des coefficients. *Journal de Mathématiques Pures et Appliquées*, 2, 56–105.
- [9] Moses, J. (1971). Symbolic integration: The stormy decade. *Communications of the ACM*, 14(8), 548–560.
- [10] Müller, N. T. (2001). The iRRAM: Exact arithmetic in C++. In *Computability and Complexity in Analysis*, Lecture Notes in Computer Science, vol. 2064, pp. 222–252. Springer-Verlag, Berlin.
- [11] Olver, F. W. J., Lozier, D. W., Boisvert, R. F., and Clark, C. W., eds. (2010). *NIST Handbook of Mathematical Functions*. Cambridge University Press, Cambridge.
- [12] Puiseux, V. (1850). Recherches sur les fonctions algébriques. *Journal de Mathématiques Pures et Appliquées*, 15, 365–480.

- [13] Risch, R. H. (1969). The problem of integration in finite terms. *Transactions of the American Mathematical Society*, 139, 167–189.
- [14] Rothstein, M. (1976). *Aspects of Symbolic Integration and Simplification of Exponential and Primitive Functions*. PhD thesis, University of Wisconsin-Madison.
- [15] Trager, B. M. (1976). Algebraic factoring and rational function integration. In *Proceedings of SYMSAC '76*, pp. 219–226. ACM Press.